This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

LDLB: Label-Based Dynamic Load Balance Algorithm for Data Center Network

Xiaoyan Qian, Rui Xu, Lei Han*, Xin He, Shuwen Wei, Qingsong Liu, Yan Gu, and Fu Xiao

***Abstract*—Existing switch-based load balance schemes in data center networks face a fundamental dilemma: local strategies respond quickly but cannot detect downstream congestion, while global strategies approximate network-wide views but suffer from information propagation delays. These issues prevent data flows from quickly and accurately bypassing congested links, resulting in prolonged Flow Completion Time (FCT). We propose Label-Based Dynamic Load Balance (LDLB), a distributed algorithm that bridges this gap by combining instantaneous local congestion detection with globally synchronized congestion labels. LDLB uses Random Early Detection and Explicit Congestion Notification (RED/ECN) to identify congested egress ports and records them in a local congestion table (LCT). Congested switches proactively generate congestion labels—identifying affected destination IPs—and forward them to upstream neighbors, which cache the labels in a congestion label table (CLT). A compressible information table aggregates contiguous IP addresses to reduce storage and synchronization overhead. Upon packet arrival, each switch consults both the LCT and CLT and applies a Weighted Cost Multi-Path (WCMP)-like cost function to reroute traffic away from congested links. Experiments on ns-3 with four real data center workloads (Web Server, Cache Follower, Web Search, Data Mining) demonstrate that LDLB reduces FCT by 57%, 61%, 65%, 67%, and 54% compared with ECMP, DRILL, LetFlow, HULA, and CONGA, respectively.**

***Index Terms*—Data Center Networks, Routing Algorithm, Load Balance, Congestion Detection.**

I. Introduction

With the advancement of network infrastructure, data center link bandwidth has continuously increased to support diverse and high-intensity workloads. Despite this ongoing bandwidth growth, the evolution of applications poses increasing challenges to network throughput, latency, and scheduling efficiency[28], [29], [31]. Applications such as cloud computing, real-time video, large-scale storage, and large language models[1], [2], [3], [4] may generate bursty flows, which require substantial bandwidth resources, sometimes even reaching the NIC's capacity. When the network forwards such flows, if the switch's egress link bandwidth is insufficient, it can result in buffer accumulation and even buffer overflow. These sudden congestion issues present a new challenge in the design of load balance algorithms for data centers. Existing distributed load balance schemes in data centers can be broadly categorized into local strategies and global strategies. Local load balance [6], [7], [9], [10] is typically deployed on network switches and is capable of making fast, real-time routing decisions. However, due to their limited visibility into downstream network conditions, they struggle to detect and avoid congestion at downstream switches. In contrast, global load balance [11], [45] performs path selection based on global congestion information, offering a broader network perspective. Nevertheless, the propagation of congestion information incurs non-negligible delays, making it difficult to react promptly to sudden congestion and thereby compromising the timeliness and effectiveness of routing decisions. Table I summarizes representative distributed load balance strategies and analyzes the challenges they face in scenarios with bursty congestion.

ECMP[6] distributes traffic randomly over equal-cost paths using a hash algorithm, effective for low bandwidth usage. However, it suffers hash polarization [5] during sudden congestion, causing uneven traffic and performance degradation. DRILL[9] performs load balance at the packet level with fine granularity, but it requires frequent contention for limited link resources and lacks the ability to perceive downstream congestion. LetFlow[10] adjusts flow slice sizes solely based on the switch's sending rate (bandwidth) and lacks congestion-aware ability, rendering it ineffective in handling congestion caused by bursty traffic. CONGA[11] is a congestion-aware load balance scheme that avoids congested links based on global congestion information. However, due to delays in the transmission of congestion information, it cannot respond promptly to sudden congestion. HULA[45] is a congestion-aware load balance scheme that uses probe packets to gather global congestion information and avoid congested links. However, probe packet propagation delays hinder its ability to respond quickly to congestion changes.

According to the above analysis, current switch-based distributed load balance strategies struggle to effectively handle sudden congestion in data center networks. To address this challenge, we propose LDLB. This approach aims to resolve the following key issues: (1) Data center networks carry massive and dynamically changing traffic. In scenarios with bursty traffic, link bandwidth fluctuates drastically. Thus, how to quickly and accurately obtain the global congestion state is critical for maintaining effective load balance; (2) Switches have limited buffer resources, raising the need to design a compressible congestion information table that reduces buffer

This work is supported by National Key Research and Development Program of China under grant 2024YFB2906704. This work is supported in part by the National Natural Science Foundation of China under Grant No. 62202237, Grant No.62372248 and Grant No.U2033207. This work is supported in part by the Key Program of Natural Science Foundation of Jiangsu under grant No.24KJA520006. This work is supported by China National Science and Technology Major Project No.2026ZD1307700.(Corresponding Author:Lei Han.)

X. Qian and R. Xu are with the College of Civil Aviation at Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China.

L. Han, F. Xiao and X. He are with the School of Computer, Nanjing University of Posts and Telecommunications, Nanjing 210023, China.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

TABLE I
ISSUES OF DISTRIBUTED LOAD BALANCE SOLUTIONS IN NETWORKS WITH SUDDEN CONGESTION

Position	Congestion Information	Existing Schemes	Issues in bursty congestion networks
Local	Blind	ECMP	Routing collision due to hash algorithm
		LetFlow	Routing collision due to congestion unawareness
Global	Congestion aware	DRILL	Throughput degradation due to packet competition and downstream congestion unawareness
		CONGA	Throughput degradation due to leaf-to-leaf congestion information delay
		HULA	Throughput degradation due to probes propagation delay

overhead and improves synchronization efficiency; (3) It is essential to develop a congestion-aware traffic allocation mechanism to avoid congested links and ensure high bandwidth with low latency.

LDLB is designed specifically for bursty congestion scenarios in data centers. Its core idea lies in combining instantaneous local congestion feedback with globally synchronized congestion labels, enabling switches to avoid congested links and nodes in a timely and accurate manner. The main contributions of this paper are as follows:

- A port-based congestion labeling mechanism is designed, along with a global congestion synchronization mechanism, enabling rapid updating and dissemination of congestion status.
- A compressible congestion information table is proposed to efficiently compress congestion information during synchronization, reducing switch buffer consumption.
- A combined local and global label-based routing mechanism is constructed, where switches update their routing tables based on both LCT and CLT. WCMP is employed to ensure load balance and achieve fast, congestion-free routing.
- LDLB is evaluated using the ns-3 network simulator with real workloads[16], [20], [26], [27]—including Web Server, Cache Follower, Web Search, and Data Mining. Experimental results demonstrate that LDLB effectively mitigates bursty congestion, reducing FCT by 57%, 61%, 65%, 67%, and 54% compared with ECMP, DRILL, LetFlow, HULA, and CONGA, respectively.

The structure of this paper is as follows: Section II presents the background and motivation. Section III describes the design of LDLB. Section IV evaluates the performance of LDLB. Section V discusses the related work. Section VI concludes the paper.

II. BACKGROUND AND MOTIVATION

A. Background

Modern data centers adopt multi-rooted topologies such as fat-tree and leaf-spine[17], [18], providing rich path diversity between any host pair. However, this redundancy introduces a fundamental challenge: how to split traffic across equal-cost paths so that no single link becomes a bottleneck. Various works have tackled this problem from different angles — centralized control, host-based steering, and in-network intelligence — each trading off congestion awareness, reaction speed, and resource overhead. We survey these works and summarize them in a unified Design Space Map (Fig. 1).

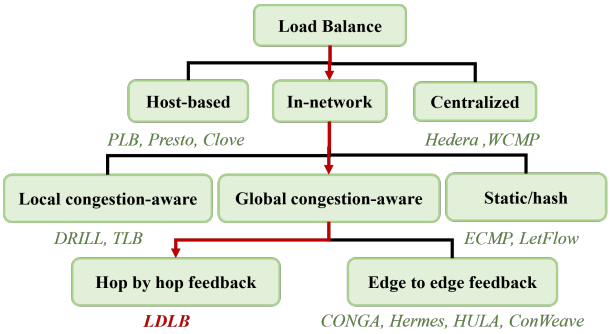


Fig. 1. Load Balance Design Space

Centralized approaches rely on a global controller to compute routing decisions. Hedera[14] dynamically schedules elephant flows via centralized multi-commodity flow optimization, but suffers from scalability limitations due to NP-hard computation and control-plane latency. WCMP[8] extends ECMP with unequal path weights but shares the same centralization limitation. MicroTE[49] adjusts routing based on short-term traffic matrices. In general, centralized schemes can theoretically achieve near-optimal placement but are constrained by control-plane latency and single-point-of-failure risks, making them poorly suited for sub-millisecond reaction times.

Host-based approaches shift routing intelligence to end hosts. PLB[60] shows that simple ECN and retransmission signals suffice for end-host load decisions without switch modifications. Presto[12] splits traffic into fixed-size units via a congestion-aware scheduler. Clove[47] adjusts per-path traffic fractions based on measured congestion. However, end hosts observe only their own traffic, lacking visibility into cross-

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

traffic congestion, which leads to suboptimal and oscillatory decisions when many hosts react simultaneously.

In-network approaches embed load balance intelligence within switches, enabling microsecond-scale reactions. ECMP[6], the de facto baseline, uses a static hash over flow five-tuples to map each flow to a fixed forwarding port. It is simple and stateless but suffers from hash collisions. LetFlow[10] randomly rehashes flowlets to different paths, but can only respond after a new flowlet forms, limiting its effectiveness under bursty traffic. DRILL[9] samples two candidate output ports per packet and forwards to the shorter queue, requiring no protocol modifications; however, multiple switches may simultaneously choose the same path, creating downstream congestion. TLB[48] similarly uses local queue occupancy for per-packet spraying with reordering mitigation at the receiver. These local schemes react quickly but are limited by their narrow view, as a switch may inadvertently push traffic toward an unobservable downstream bottleneck.

Global congestion-aware schemes exchange information across switches to approximate a network-wide view. CONGA[11] maintains congestion-to-leaf and congestion-from-leaf tables at each leaf switch, with remote feedback piggybacked on return traffic; however, it introduces significant state overhead and detection delays that limit scalability. Hermes[46] improves upon CONGA by incorporating path diversity. HULA[45] propagates link utilization through probe packets, reducing per-destination memory, but its probe dissemination introduces propagation latency. ConWeave[59] targets RDMA by performing per-flow RTT monitoring at the source ToR for congestion detection, constraining each flow to at most two active paths, and reordering packets at the destination ToR via queue pausing. LDLB takes a different approach by combining instantaneous local congestion feedback with globally synchronized congestion labels. Each switch maintains an LCT from its own port states and a CLT populated through global synchronization, with a compressible information table to reduce overhead. Switches update routing based on both LCT and CLT and employ WCMP to ensure balanced load distribution.

Despite this rich design space, existing in-network load balance schemes still face a fundamental dilemma among congestion blindness, state overhead, and reaction latency. Static schemes such as ECMP and LetFlow are lightweight but blind to congestion. Local schemes such as DRILL and TLB respond quickly with minimal state, yet are blind to downstream congestion. Global schemes such as CONGA and HULA approximate a network-wide view but pay the cost in per-destination state or probe latency. ConWeave achieves fine-grained RDMA rerouting but requires per-flow reordering queues and RTT monitoring. No existing solution simultaneously achieves fast reaction, low state overhead, and timely global congestion awareness — a gap that motivates our work.

B. Motivation

*1) **Motivation for Designing Congestion-Aware Load Balance Strategy**:* To illustrate the hash polarization problem inherent in randomized algorithms, we conduct an ECMP path-selection simulation. The 5-tuple (source IP, destination IP, source port, destination port, and protocol) of 1000 flows are hashed and mapped to four switch egress ports via a modulo operation. The statistical results, shown in Fig. 3(a), indicate that the flow distribution across the egress ports is uneven: egress 2 carries the largest number of flows, while egress 4 carries the fewest. This outcome stems from the fundamental property that random processes follow a normal distribution. In real networks, flow sizes are typically non-uniform, and switch bandwidth is constrained by various physical factors. Consider an extreme scenario: if the physical bandwidth of egress 2 is severely limited and a long flow happens to be hashed to that port, severe congestion will occur at that egress. This phenomenon, where uneven hash mapping leads to overload on specific egress ports, is referred to as hash polarization.

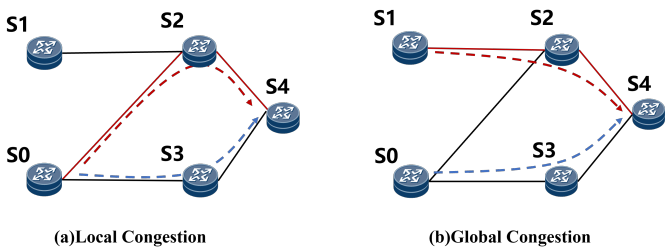


Fig. 2. The location of local and global congestion within a network.

To further demonstrate that algorithms lacking congestion awareness can degrade throughput, we design a small-scale hash-polarization validation experiment based on the topology in Fig. 2. In this experiment, a host under switch s0 sends 50 short flows (5KB each) and one long flow (100MB) toward s4 at an aggregate rate of 100Gbps. To ensure that the long flow dominates bandwidth usage, the link between s0 and s2 is limited to 1Gbps, while all other links remain at 100Gbps. We compare the conventional ECMP with the congestion-aware LDLB (Local) algorithm. LDLB (Local) can dynamically detect and temporarily disable congested paths, then re-apply ECMP hashing over the remaining available paths. As shown in Fig. 3(b), conventional ECMP, being unaware of congestion, fails to efficiently utilize uncongested links when path collisions occur, resulting in a significant throughput drop. This result clearly underscores the necessity of incorporating traffic deflection and congestion awareness into load-balance mechanisms.

*2) **Motivation for Designing Global + Local Label Routing Mechanism**:* In networks, congestion caused by bursty traffic and bandwidth constraints can be divided into local congestion and global congestion. Local congestion occurs on the egress link of the current network node and can be directly detected and mitigated via local measurement information. By contrast, global congestion takes place on downstream links far beyond the current node instead of its direct egress. As the congestion location is outside the local scope, the node has to perceive such congestion with the help of external signaling. Without such signaling assistance, data traffic will continue to be forwarded along paths that are locally idle but downstream congested, resulting in overall network performance degradation.

As shown in Fig. 2(a), from the perspective of switch s0,

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

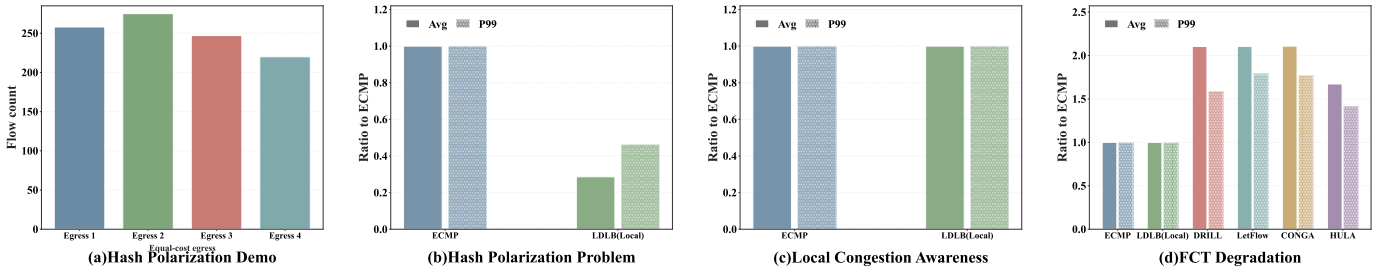


Fig. 3. (a) ECMP algorithm path selection simulation. (b) Using the topology of Fig. 2 for a small-scale experiment, explain the issue of hash polarization. (c) Using the topology of Fig. 2 for a small-scale experiment, explain the issue with the local congestion-aware algorithm. (d) Using 2×3 leaf-spine topology for a small-scale experiment, explain the problem of FCT degradation in distributed load balance schemes in bursty congestion scenarios.

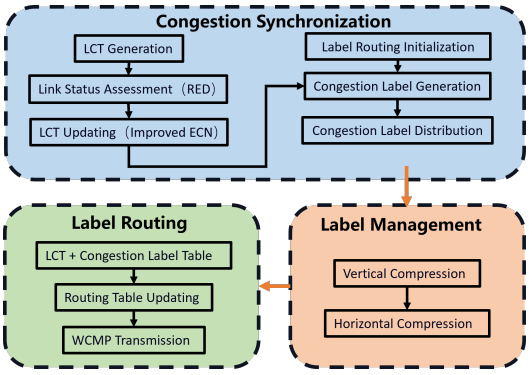


Fig. 4. LDLB Algorithm Framework

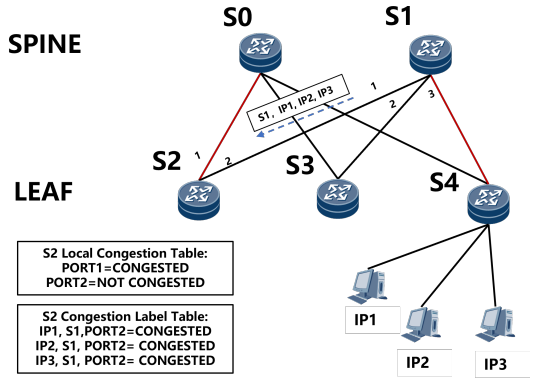


Fig. 5. Local and global congestion synchronization.

when a long flow traverses the path S0→S2→S4 and causes path congestion, it constitutes local congestion. LDLB (Local) can detect the congested link caused by the long flow, and subsequent flows are rerouted to S0→S3→S4. However, in Fig. 2(b), when a long flow traverses S1→S2→S4 and induces congestion along that path, the situation represents global congestion. If only LDLB (Local) is employed, switch S0 cannot perceive the congestion on path S2→S4, and a portion of the data flows originating from S0 will still choose S0→S2, thereby entering the downstream congested link.

We conduct a small-scale experiment using the topology in Fig. 2. Switch s1 transmits a 100MB long flow at 100Gbps, while s0 sends 50 short flows of 5KB each at 100Gbps; both streams are destined for s4. The link between s2 and s4 is configured as 1Gbps, whereas all other links operate at 100Gbps. As shown in Fig. 3(c), in this scenario LDLB (Local) is unable to sense the downstream congestion, and its cumulative distribution function (CDF) curve coincides completely with that of ECMP. In some cases, LDLB (Local) even performs worse than ECMP; a detailed analysis of this will be provided in the experimental evaluation in Section IV. Consequently, we designed a global+local label-routing mechanism.

*3) **The issue of FCT degradation in switch-based distributed load balance algorithms**:* This paper also conducts a 2×3 bursty congestion scenario. A host under one leaf switch simultaneously transmits 50 short flows (5KB each) and 1 long flow (100MB) at 100Gbps to a host under another leaf switch. The leaf-spine link bandwidth is configured as 10Gbps. We compare the performance of LDLB (Local) with five other distributed load-balance algorithms; their limitations are summarized in Table I. As shown in Fig. 3, LDLB (Local) delivers comparable average and P99 FCT to ECMP, whereas all other schemes exhibit noticeable FCT degradation, with their achievable throughput even halved relative to ECMP. The experimental results in Section IV will further elaborate on this issue.

III. LDLB Design

A. Overview

LDLB is a distributed, in-network load balance algorithm that combines local congestion detection with globally synchronized congestion labels to avoid both local and downstream congested links. The overall framework is shown in Fig. 4. We illustrate its workflow through the example shown in Fig. 5.

Consider a leaf-spine topology where switch $S2$ forwards traffic toward a destination host under $S4$ via two spine switches, $S0$ and $S1$. Suppose the egress link $P3$ on $S1$ becomes congested due to a bursty flow. LDLB operates in three main stages:

Stage 1 — Local congestion synchronization. $S1$ monitors its egress queue lengths using RED/ECN. When the queue on $P3$ exceeds the congestion threshold, $S1$ marks $P3$ as congested in its LCT. At this point, $S1$'s local forwarding can already avoid $P3$ for new flows.

Stage 2 — Global congestion synchronization. However, $S2$ is unaware of this downstream congestion and may continue forwarding traffic toward $S1$. To address this, $S1$ generates a *congestion label* identifying the affected destination IPs and the congested port, and sends it to its direct neighbor $S2$. Upon receiving the label, $S2$ caches it in its CLT. Now

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

TABLE II
SUMMARY OF NOTATIONS

Notation	Meaning
K	The marking threshold
C	The link capacity
RTT	The round-trip times
λ	The parameter determined by congestion control algorithms
K_{max}	The high buffer occupancy threshold
K_{min}	The low buffer occupancy threshold
N	The number of flows or sources in the network
Avg	The average queue length at the previous time point
ω_q	The weight factor that controls the influence of the newly arrived queue length on the average value
P_b	The initial packet-marking probability
P_{max}	The maximum value for the packet marking probability
Q	The actual queue length at the current moment
P	The final packet marking probability
X	The inter marking time
F	The incoming flow
f_i	The flow assigned to the i-th path
P_i	The path index / path that determines which path P_i the flow will follow
$h(flow)$	The hash function used to map the flow to a path

$S2$ knows that traffic destined to certain IPs should avoid the path through $S1$. This label propagation can further traverse hop-by-hop upstream, allowing all switches along the path to react.

Stage 3 — Label-based routing. When a packet arrives at $S2$, LDLB checks both local (LCT) and global (CLT) states. If the destination IP is marked in the CLT as impacted by congestion on $S1$'s $P3$, $S2$ first withdraws the corresponding egress port from use. It then evaluates candidate paths using a WCMP-like cost function and selects one whose egress port on $S2$ is free of congestion (e.g., the path through $S0$). For unaffected destinations, or when all candidate paths are congested and the system falls back to default behavior, traffic follows standard ECMP.

The remainder of this section details each stage. Section III.B describes the congestion assessment and synchronization mechanism (Stages 1–2). Section III.C presents the label compression technique that keeps the CLT small. Section III.D details the label-based routing decisions and stability mechanisms (Stage 3).

B. *Congestion Synchronization*

In data center networks, timely detection and synchronization of congestion information are critical for improving load balance efficiency and optimizing link resource utilization. The primary objective of the congestion synchronization mechanism is to enable real-time detection, recording, and dissemination of local link congestion states, thereby guiding flow routing decisions and preventing new traffic from being assigned to already congested paths. The notation and symbols used in this paper are shown in Table II.

*1) **Congestion Assessment**:* Continuing with the example in Fig. 5, suppose the egress queue on $S1$'s port $P3$ begins to grow due to a bursty flow. The first question LDLB must answer is: *has $P3$ become congested?* This section presents the RED/ECN-based method for evaluating link congestion degree and computing marking thresholds and probabilities. The ECN marking threshold K has the most significant impact on network performance. Unlike the internet, data center environments typically exhibit a lower number of concurrent large flows[22], [23]—approximately only 2 flows at the 75th percentile. Given this limited statistical multiplexing, this paper considers an ideal scenario where several synchronized, long-lived flows share a bottleneck link with bandwidth C, and these flows have the same round-trip time (RTT). As previous research[24] has indicated, to maximize link capacity and maintain low latency, the ECN marking threshold K should be configured as follows:

$$K = C \times \mathrm{RTT} \times \lambda \tag{1}$$

where λ is a parameter determined by the congestion control algorithm (e.g., for ECN, $\lambda = 1$[23]). In this paper, $C \times \mathrm{RTT} \times \lambda$ is referred to as the standard marking threshold for ECN.

Previous experience has shown[23] that calculating the standard marking threshold in data centers is feasible. This is because, unlike the internet, the RTT in data center networks is relatively stable and can be well estimated through large-scale measurements[19]. For example, in Microsoft's data centers, 23%, 74%, and 90% of the inter-rack RTTs are approximately 200 microseconds, 300 microseconds, and 400 microseconds, respectively[23]. Additionally, since operators have full control over the network stacks of end hosts, the value of λ can be easily obtained. Therefore, calculating such a static standard marking threshold in production data centers is quite straightforward.

RED Packet Marking Probability: In current switch chips, ECN is typically used in conjunction with RED, which utilizes average buffer occupancy as a congestion signal. RED has at least three configurable parameters: the low buffer occupancy threshold K_{min}, the high buffer occupancy threshold K_{max}, and the packet marking probability P. Based on the analysis of K in the DCTCP paper, K_{max} and K_{min} are derived as follows:

$$K_{\max} = K + N, \tag{2}$$

$$K_{\min} = K + N - \frac{1}{2}\sqrt{2N(C \times \mathrm{RTT} + K)}, \tag{3}$$

here, N represents the number of concurrent long-lived flows on a specific port/bottleneck link. Each flow has its own transmission window, which jointly influences the overall queue dynamics. The number of concurrent long-lived flows on a port is critical for calculating the system's overall behavior, especially when managing congestion and queue fluctuations in multi-source scenarios. Existing research has proposed various methods for estimating N, including SYN/FIN packet-counting[30] (largely dismissed due to insufficient accuracy), complex online estimation algorithms (rarely adopted because of high overhead), and the approach of querying the OpenFlow flow table via an SDN controller. In our analysis and modeling, we do not adopt the above real-time measurement schemes. Instead, we directly employ the empirical value obtained from the DCTCP paper's large-scale measurement of data-center traffic, i.e., the 75th percentile of the number of concurrent large flows (>1 MB) on a server port is 2. This value reflects the "low statistical multiplexing" characteristic of data-center networks and is widely used as a reasonable empirical parameter in related performance analyses.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

Before computing the packet marking probability, this paper also requires the average queue length. In RED, the average queue length is typically calculated using the Exponentially Weighted Moving Average (EWMA) model. The basic formula is as follows:

$$\text{Avg} = (1 - \omega_q) \times \text{Avg} + \omega_q \times Q, \tag{4}$$

here, Avg is the average queue length at the previous time point, Q is the actual queue length at the current moment, and ω_q[21] is the weight factor, which controls the influence of the newly arrived queue length on the average value. It is typically chosen within the range of $0 < \omega_q \leq 1$.

There are two methods to calculate the final packet marking probability: the geometric random variable method and the uniform random variable method. In the first method, when the average queue size remains constant, the number of packets between arriving marked packets follows a geometric distribution. In the second method, the number of packets between arriving marked packets follows a uniform distribution. This paper adopts the second method. The initial packet marking probability P_b is calculated as a linear function of the average queue size.

$$P_b = \frac{P_{\max}(Avg - K_{\min})}{K_{\max} - K_{\min}}, \tag{5}$$

the parameter P_{max} gives the maximum value for the packet marking probability P_b, achieved when the average queue size reaches the maximum threshold.

Using the uniform random variable method, we obtain the final packet marking probability P. The formula is as follows:

$$P[X = n] = \frac{P_b}{1 - (n-1)P_b} \prod_{i=0}^{n-2} \left(1 - \frac{P_b}{1 - iP_b}\right), \tag{6}$$

$$= P_b \quad \text{for } 1 \leq n \leq \frac{1}{P_b}, \tag{7}$$

$$\text{And} \quad P[X = n] = 0 \quad \text{for } n > \frac{1}{P_b} \tag{8}$$

$$\text{For this method, } E[X] = \frac{1}{2P_b} + \frac{1}{2}. \tag{9}$$

let the inter marking time X to be a uniform random variable from $\{1, 2, \ldots, 1/P_b\}$ (assuming, for simplicity, that $1/P_b$ is an integer). This is achieved if the marking probability for each arriving packet is $P_b/(1 - \text{count} \times P_b)$, where count is the number of unmarked packets that have arrived since the last marked packet.

*2) **Local Congestion Synchronization**:* Returning to our example: once RED determines that $P3$ is congested, $S1$ must record this state for use in local routing decisions. As shown in Fig. 5, during topology table configuration, unique labels are assigned to each switch and its ports, and the connection between the two switches at both ends of a link is established. The ECN mechanism is modified to shift from flow-based congestion marking to port-based congestion marking. The congestion status of switch ports is updated in real time and cached in the LCT. Its format is shown in Table III, which includes the current switch ID and the congestion state of each port, where "C" indicates congestion and "0" indicates no congestion.

TABLE III
LOCAL CONGESTION TABLE

Switch ID	Port1 State	Port2 State
$S2$	$P1$=C	$P2$=0

TABLE IV
CONGESTION LABEL

Switch ID	Unreachable Dst IP Address		
$S1$	$Dest\ IP1$	$Dest\ IP2$	$Dest\ IP3$

To avoid triggering the rate control mechanism at the sender, the algorithm proposed in this paper does not modify the Differentiated Services Code Point (DSCP) field [25] in the packet header. Instead, it records the label of the congested port in the LCT within the switch's Memory Management Unit (MMU). The collaborative performance of DCTCP combined with LDLB has also been evaluated, with the results and analysis detailed in Section IV.

As shown in Algorithm 1, the mechanism determines whether congestion has occurred by checking if the queue byte count exceeds the maximum threshold or by evaluating the congestion level based on the final packet marking probability p. First, it checks whether the queue byte count $E[P]$exceeds the maximum threshold $k_{max}[P]$. If so, it returns true, indicating that congestion has occurred. If the byte count exceeds the lower threshold $k_{min}[P]$, it calculates the congestion probability p. If a randomly generated value is less than p, it likewise returns true, signifying that congestion is likely. Subsequently, the congested egress port is recorded in the switch's LCT, in a format similar to an entry in a routing table. Once the port recovers, its port state is updated accordingly.

*3) **Global Congestion Synchronization**:* The key question now is: *how does $S2$ learn that $S1$'s $P3$ is congested?* LDLB lets $S1$ proactively generate a congestion label and send it to its upstream neighbor $S2$. Specifically, $S1$ queries its routing table to find all destination IPs whose forwarding path traverses $P3$ (the affected IPs), packages them along with $S1$'s identifier and $P3$'s congestion state into a congestion label (format shown in Table IV), and sends it to $S2$.

In congestion-aware load balance algorithms, the method of propagating congestion information plays a decisive role in overall system performance. Global load balance algorithms employ an indirect feedback mechanism: congestion information must first be carried by forward packets from the source switch to the destination switch, and then returned to the source via reverse-direction packets. This round-trip feedback path introduces significant latency, limiting the algorithm's ability to respond rapidly to sudden congestion. To mitigate this issue, the mechanism proposed in this paper allows each switch to directly generate congestion labels based on its LCT and routing table, and proactively forwards them to neighboring switches. This enables distributed, one-hop direct

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

Algorithm 1 Local Congestion Synchronization Algorithm. P refers to the port label. E represents the egress queue length, while C is a boolean flag indicating whether the port is congested. The LCT records the congestion status of ports. The congestion thresholds are denoted by k, and p is the final packet marking probability.

function PORTCONGESTIONDETECT(P)
  if $E[P] > k_{max}[P]$ **then** ▷ upper bound
    return $C[P] \leftarrow$ congested
  else if $E[P] \leq k_{min}[P]$ **then** ▷ lower bound
    return $C[P] \leftarrow$ not congested
  else if $E[P] > k_{min}[P]$ **then** ▷ probabilistic region
    $p \leftarrow p_{max}[P] \cdot \frac{E[P] - k_{min}[P]}{k_{max}[P] - k_{min}[P]}$
    if uniform random value $< p$ **then** ▷ marking
      return $C[P] \leftarrow$ congested
    end if
  end if
end function
procedure SETCONGESTIONLABEL
  for each port P in switch **do**
    PORTCONGESTIONDETECT(P)
    if $C[P] =$ congested **then** ▷ congested port
      if $P \notin LCT$ **then**
        append P to LCT
      end if
    else if $C[P] =$ not congested **then** ▷ normal port
      if $P \in LCT$ **then**
        remove P from LCT
      end if
    end if
  end for
end procedure

propagation of congestion information, significantly reducing the end-to-end latency associated with traditional reverse-path feedback.

The core function of a congestion label is to inform its neighboring switches which destination IP addresses have no available next-hop at the current switch. The format of the congestion label is shown in Table IV, and consists of two main parts: the unique identifier of the switch that generated the label, and the list of destination IP addresses that are currently unreachable due to congestion.

Algorithm 2 describes the process of generating and forwarding congestion labels. The algorithm consists of two core procedures. FindIPs takes a congested port P and collects destination IPs whose all next-hop ports are congested. ForwardCL monitors LCT state changes and sends the appropriate label to upstream neighbor B.

When ForwardCL detects a state change, it handles two cases differently. Upon congestion onset (non-congested $\rightarrow$ congested), it calls FindIPs to obtain the affected BlockedIPs list and sends a label only if this list is non-empty, avoiding unnecessary control traffic. Upon congestion relief (congested $\rightarrow$ non-congested), it proactively sends a relief label with an empty BlockedIPs list to the upstream neighbor B, signaling port recovery. This ensures that upstream switches clear stale

Algorithm 2 Global Congestion Synchronization Algorithm. The inputs are the source switch A, neighbor switch B, LCT, and the routing table RT. Key variables are: the port P (iterating over all ports in LCT), the blocked-IP list $BlockedIPs$ (collecting IPs unreachable via P), and the congestion label CL.

function FINDIPS(P, RT, LCT)
  AffectedIPs $\leftarrow []$
  for each entry $destIP$ in RT **do**
    if $P \in$ nextHop($destIP$) **then**
      allCongested $\leftarrow$ True
      for each port Q in nextHop($destIP$) such that $Q \neq P$ **do**
        if $LCT[Q]$.CurrentState $\neq$ congested **then**
          allCongested $\leftarrow$ False
          break
        end if
      end for
      if allCongested $=$ True **then**
        AffectedIPs.append($destIP$)
      end if
    end if
  end for
  return AffectedIPs
end function
procedure FORWARDCL(A, B, LCT, RT)
  for each port P in LCT **do**
    if $LCT[P]$.PreviousState $\neq LCT[P]$.CurrentState **then**
      if $LCT[P]$.CurrentState $=$ congested **then**
        BlockedIPs $\leftarrow$ FindIPs(P, RT, LCT)
        if BlockedIPs $\neq \emptyset$ **then**
          $CL \leftarrow$ CreateLabel(A.ID, BlockedIPs)
          SetPriority($CL, 0$)
          SendLabelTo(B, CL)
        end if
      else
        $CL \leftarrow$ CreateEmptyLabel(A.ID)
        SetPriority($CL, 0$)
        SendLabelTo(B, CL)
      end if
    end if
  end for
end procedure

CLT entries immediately. Each label carries a TTL field (default $\mathcal{T} = 2$) that is decremented per hop to bound the propagation scope; labels with TTL $= 0$ are not forwarded further. As a safety net against lost relief labels, each CLT entry also has an aging timeout $\tau_{expire} = 10$ ms; if no refresh or new label arrives within this window, the entry is automatically purged.

C. *Label Management*

Upon receiving a congestion label, $S2$ must parse and store it in its CLT. In our example, the CLT generated by $S2$ after receiving the label from $S1$ is shown in Table V: traffic destined for $Dest\ IP1/2/3$ traverses $S1$'s $P3$, which is

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

TABLE V
CONGESTION LABEL TABLE

Blocked IP	Congested Switch	Related port
$Dest\ IP1$	$S1$	$P2 = C$
$Dest\ IP2$	$S1$	$P2 = C$
$Dest\ IP3$	$S1$	$P2 = C$

currently congested and therefore disabled. In supercomputing data centers, there exist tens of thousands of host nodes, and congestion labels are typically composed of destination IP addresses. When a large number of links become congested, the corresponding labels inevitably contain extensive destination IP information, leading to additional bandwidth overhead. Furthermore, due to limited memory capacity in switches, it is impractical for the congestion label table to cache an unbounded number of entries. To address these issues, we design a compressible congestion label and label table structure.

*1) **Format of the Congestion Label Table**:* The congestion label table needs to reorganize congestion labels. The congestion information table is divided into three columns, including blocked IP, congested switch ID, and port number of the current switch connected to the congested switch (the port number is derived from the topological relationship). Taking Fig. 5 as an example, the congestion label table generated by S1 and received by S2 is shown in Table V. The second row in the table indicates that for packets forwarded by S2 with destination IP1, S1 is the congested switch, and the outgoing link P2 connected at S1 is considered congested and therefore disabled.

*2) **Vertical Compression of Label Table**:* In a real data center, a spine switch may serve hundreds of hosts. If $S1$'s $P3$ is congested and affects 256 destination IPs, the congestion label would need to list all 256 entries, and the CLT would require 256 rows—this is impractical. Fortunately, hosts under the same switch in data centers typically share a common IP prefix with contiguous suffixes. LDLB exploits this property for compression: the compression of label tables and congestion labels is fundamentally aligned, with its core lying in the efficient aggregation of IP address representations. In data center networks, the IP addresses of hosts connected to switches typically exhibit significant prefix similarity and suffix continuity. In response to this characteristic, this paper proposes a compression model based on interval representation: aggregating m IPv4 addresses that share the same next hop and have contiguous suffixes from individual enumeration storage into a single table entry in the format of "common prefix + minimum suffix + maximum suffix." Assuming the common prefix length is k bits, the original storage requires n independent entries, while after compression, only one entry is needed. The reduction ratio in the number of entries is $1-\frac{1}{n}$, approaching 100% when n is large. The storage overhead per entry also decreases from a fixed $32n$ bits to a constant $k+2\times(32-k) = 64-k$. The overall compression ratio can be quantified as $R = \frac{32n}{64-k}$. In a typical scenario (k=24, n=256), the compression ratio can reach 205 times, equivalent to saving approximately 99.5% of bandwidth and storage space, while reducing the number of label table entries from n to 1, significantly alleviating the entry pressure on switch TCAM or SRAM. This model maintains the integrity of congestion information while achieving a substantial reduction in control-plane overhead in both the number and size of entries, making it particularly suitable for large-scale data center environments where IP addresses are allocated in contiguous blocks.

In Section IV, we evaluate the performance based on a typical 12×8 Leaf-Spine network, where each Leaf switch connects 32 hosts. In the uncompressed scenario (equivalent to all IP addresses being non-contiguous), if all host IPs (32 addresses in total) within a complete /24 subnet under a port need to be announced due to congestion, each IPv4 address occupies 4 bytes, resulting in a single congestion label size of 32 × 4 = 128 bytes. Under high-load conditions with a total traffic of 6GB, the control-plane traffic accounts for only 5MB, maintaining a control-to-data traffic ratio of 0.27%, demonstrating the low-overhead nature of the proposed mechanism.

*3) **Vertical and Horizontal Compression of Label Table**:* With the increasing scale of high-performance computing in data centers, the use of IPv6 addresses is becoming a trend. For IPv6 addresses, label compression requires the use of Ternary Content Addressable Memory (TCAM). First, zero-compression is used to initially process the IP address. This method is particularly suitable for IPv6 addresses, as it removes consecutive zeros from the address, achieving a moderate reduction in address length. Next, prefix overlapping is used to reduce the vertical entries in the congestion label table. Finally, prefix minimization is introduced, which employs the Espresso-II algorithm to achieve deeper compression. Prefix minimization focuses on analyzing the prefix information in IP addresses, identifying and removing redundant parts, thereby further optimizing the size of IP addresses. This comprehensive compression strategy not only improves storage efficiency but also facilitates subsequent network processing and data transmission.

D. Label Routing

At this point, $S2$ has local congestion awareness through its LCT and downstream congestion awareness through its CLT. The final question is: *how does $S2$ use both tables to make routing decisions?* Before detailing the routing logic, we first describe two initialization components that ensure robust operation.

The label-based routing mechanism is a core component of LDLB. To address congestion at different locations in the network, we design a label-routing strategy that handles both local and global congestion.

*1) **Label-Routing Initialization**:* Before label-based decisions are executed, the network performs initialization and state preparation to establish a consistent network view and incorporate fault-tolerance and anti-oscillation safeguards. This process includes two components:

Failure Awareness: Each switch is preconfigured with a static topology table mapping the local switch ID, downstream neighbor IDs, and corresponding local egress ports (see Table VI). This table is the basis for identifying physical links.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

TABLE VI
TOPOLOGY TABLE

Current Switch ID	Downstream Switch ID	Link Number
$S2$	$S0$	1
$S2$	$S1$	2

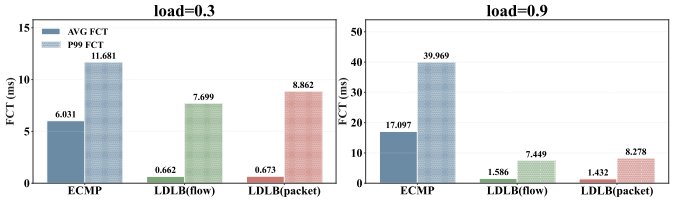


Fig. 6. The performance comparison of AVG and P99 FCT for the algorithms ECMP, LDLB (without flow pinning), and LDLB (with flow pinning) under load intensities of 0.3 and 0.9.

To improve hard-failure detection, the design can integrate an external fault-localization system (e.g., NetBouncer[56]). Using IP-in-IP active probing and link-failure inference[57], [58], the system continuously detects device/link failures and reports them as failure labels to the controller. The controller then disseminates these labels to relevant switches and updates their topology tables. A strict filtering rule is enforced: any link labeled as failed is immediately and permanently removed from the candidate next-hop set.

Congestion-delay Timer: To mitigate transient inconsistency between local and global congestion states, we introduce a congestion-delay timer at core switches. During the interval between local detection and global synchronization, upstream switches may still forward traffic toward newly congested core switches, potentially causing transient drops due to unavailable next hops. Therefore, a congested link is not disabled immediately. Instead, a delay timer is started, with duration determined by the ratio of congestion-label size to link bandwidth. During this window, the link remains temporarily available, allowing time for global label propagation and synchronization, thereby reducing synchronization-induced packet loss.

*2) **Local Label Routing**:* Consider a local congestion scenario: one of $S2$'s own egress ports P_x becomes congested (marked as "C" in the LCT). $S2$ simply queries the LCT before applying ECMP hashing, temporarily removing P_x from the candidate port set so that new flows are automatically directed to other uncongested ports. ECMP, a widely used local load-balance method, distributes flows by applying a hash-modulo operation to the flow five-tuple:

$$i = h(\text{flow.fivetuple}) \bmod k, \tag{10}$$

where $h(\text{flow.fivetuple})$ denotes the hash function, i is the selected path index, and k is the number of candidate paths.

This strategy can suffer from hash collisions and uneven hash-value distribution. As a result, multiple flows may be mapped to the same path, overloading some links while leaving others underutilized. The problem is particularly pronounced under local congestion, where ECMP may continue forwarding traffic onto already congested paths.

Our mechanism mitigates this issue by consulting the switch-maintained LCT before final path selection. Ports marked as congested are temporarily removed from the routing table and are restored once congestion is cleared.

To further improve stability, we employ flow pinning to avoid routing oscillation and intra-flow packet reordering caused by rapid state transitions (congested/non-congested). Each switch maintains a per-port active-flow table that records the five-tuples of currently forwarded flows. When a port becomes congested, existing flows on that port are pinned to it, ensuring in-order delivery. Only newly arriving flows are allowed to react to updated congestion states and select alternative uncongested paths. This design preserves packet ordering while retaining load-balance flexibility for new traffic.

We quantify the effect of flow pinning on the 12×8 leaf-spine topology under a mixed workload of Cache Follower and 50:1 Incast traffic, with offered loads of 0.3 and 0.9. As shown in Fig. 6, at load 0.3, LDLB (packet), which does not use these stability mechanisms, exhibits about 15% worse tail FCT than LDLB (flow). At load 0.9, LDLB (packet) achieves better average metrics but incurs approximately 1% flow loss (47 out of 7,753 flows). In some scenarios, LDLB (packet) without stability safeguards even performs worse than ECMP in both latency and loss. These results highlight the necessity of the proposed mechanisms for stable label-routing operation under dynamic, high-variance traffic conditions.

*3) **Global Label Routing** :* We now return to our core scenario: $S2$ has no local congestion, but its CLT indicates that downstream $S1$'s $P3$ is congested. When a packet destined for $Dest\ IP1$ arrives at $S2$, it queries the CLT and finds that $Dest\ IP1$ is affected by congestion on $S1$'s $P3$. $S2$ then looks up its topology table (Table VI) to find the egress port toward $S1$, temporarily disables that port for this destination, and selects an alternative path through $S0$ via the WCMP-like cost function. This is the core logic of Algorithm 3.

Due to an inherent limitation of the local label-based routing mechanism—namely, the inability of upstream switches to perceive downstream congestion states—their path selection cannot proactively avoid congested links farther along the data path. Consequently, the local mechanism can only mitigate congestion at the current switch and fails to address global congestion scenarios where upstream switches may continue to forward traffic to already congested downstream switches.

To address this gap, we propose a global label-based routing mechanism. This mechanism actively propagates congestion labels generated by congested downstream switches to all their upstream switches. Upon receiving these global labels, upstream switches can update their routing tables by integrating both local congestion information and the received global labels, thereby enabling proactive avoidance of downstream congested links.

Algorithm 3 implements the proposed global label-based routing mechanism, as outlined below: Upon receiving a packet pkt, the algorithm first extracts its destination IP address. If the IP is recorded in $CLT.BlockedIPs$—indicating downstream congestion—the algorithm identifies the congested downstream switch B via $CLT.CongestedSW(IP)$ and queries the topology table TT to obtain the corresponding egress port $CongestedPort$ from the current switch A to B. If this port is present in the set of available next hops

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

$RT.nexthops$ in the routing table, it is temporarily disabled for the destination IP, removing it as a viable forwarding option. Subsequently, the algorithm checks whether any feasible path to the destination IP remains. If a path exists, it invokes $SelectNextHop(RT, IP)$ to select a non-congested next hop and forwards the packet; otherwise, the packet is dropped via dropPkt(pkt). This process enables congestion-aware routing while preserving network connectivity whenever possible.

Algorithm 3 Global Label-based Routing. pkt is the packet to be processed, containing network-level information. A identifies the current upstream switch. TT is the topology table, where $TT(A, B)$ gives the egress port from switch A to switch B. RT is the routing table, with $RT.nexthops$ as the set of available next-hop ports.

procedure PROCESSPACKET(pkt,A,CLT,TT,RT)
  $IP \leftarrow pkt.destIP$
  if $IP \in CLT.BlockedIPs$ **then** ▷ congestion-aware
    $B \leftarrow CLT.CongestedSW(IP)$
    if $B \in TP$ **then**
      $CongestedPort \leftarrow TT(A, B)$ ▷ affected port
      if $CongestedPort \in RT.nexthops$ **then**
        Temporarily disable $CongestedPort$ for IP in forwarding logic
      end if
    end if
  end if
  if feasible path to IP exists **then**
    $Port \leftarrow$ SelectNextHop(RT, IP) ▷ path selection
    return ForwardPkt $(pkt, Port)$
  else
    return $dropPkt(pkt)$ ▷ no path
  end if
end procedure

*4) **WCMP-like Reroute**:* To enable adaptive traffic redirection over multiple candidate output ports, a WCMP-like reroute mechanism is developed based on a composite cost metric. The proposed method jointly considers queue depth, available bandwidth, and failure rate, such that rerouting decisions can capture both congestion and reliability conditions of candidate ports.

For each candidate output port i, the queue-depth cost is defined as

$$C_q(i) = \frac{q_i - q_{\min}}{q_{\max} - q_{\min} + \varepsilon}, \tag{11}$$

where q_i denotes the queue depth of port i, and $q_{\min}$ and $q_{\max}$ denote the minimum and maximum queue depths among all candidate ports, respectively.

The available-bandwidth cost is defined as

$$C_b(i) = \frac{b_{\max} - b_i}{b_{\max} - b_{\min} + \varepsilon}, \tag{12}$$

where b_i denotes the available bandwidth of port i, and $b_{\min}$ and $b_{\max}$ denote the corresponding minimum and maximum values among candidate ports. Inverse normalization is adopted so that a larger available bandwidth yields a lower cost.

The failure-rate cost is defined as

$$C_f(i) = \frac{f_i - f_{\min}}{f_{\max} - f_{\min} + \varepsilon}, \tag{13}$$

where f_i denotes the failure rate of port i, and $f_{\min}$ and $f_{\max}$ are the minimum and maximum failure rates among candidate ports, respectively.

The overall rerouting cost is then given by

$$Cost(i) = w_q C_q(i) + w_b C_b(i) + w_f C_f(i), \tag{14}$$

where w_q, w_b, and w_f denote the weights associated with queue depth, available bandwidth, and failure rate, respectively, subject to

$$w_q + w_b + w_f = 1, \qquad w_q, w_b, w_f \geq 0. \tag{15}$$

The selected output port is determined by

$$i^* = \arg\min_i Cost(i). \tag{16}$$

Therefore, the reroute decision is converted into a minimum-cost selection problem over candidate ports. Compared with single-metric or static forwarding schemes, this formulation enables more adaptive and robust traffic redirection under dynamic network conditions.

The weight parameters can be determined in several practical ways. A heuristic setting can be adopted according to the design objective, e.g., assigning a larger w_q in delay-sensitive scenarios. Alternatively, grid search can be used to evaluate feasible weight combinations over historical traces or simulation datasets and select the one with the best overall performance. The weights can also be obtained through constrained optimization by minimizing a performance objective such as average delay, packet loss, or a composite utility function. In practice, grid search is suitable for parameter initialization and prototype evaluation, while optimization-based tuning is more effective when sufficient measurement data are available.

IV. EVALUATION

A. *Experimental Setup*

This paper evaluates the performance of LDLB under both local and global congestion scenarios using the ns-3 simulation platform[61], [62], [63].This paper evaluates the performance of LDLB under both local and global congestion scenarios using the ns-3 simulation platform[61], [62], [63]. The implementation of LDLB has been open-sourced and is available on GitHub(https://github.com/xuruiiiii/LDLB-NS3).

Topology Structure: To comprehensively evaluate the proposed method under diverse network conditions, we built three data center network (DCN) scenarios on an ns-3-based flow-level simulation platform.

Topology 1 uses a leaf-spine topology with 8 spine switches and 12 leaf switches. Each leaf switch connects to 32 hosts, resulting in 384 hosts in total. The host-to-leaf access links are configured at 100 Gbps, and the leaf-to-spine uplinks are configured at 400 Gbps. To emulate different congestion levels, the bandwidth of selected links is further adjusted. **Topology 2** also adopts a leaf-spine topology with the same network

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

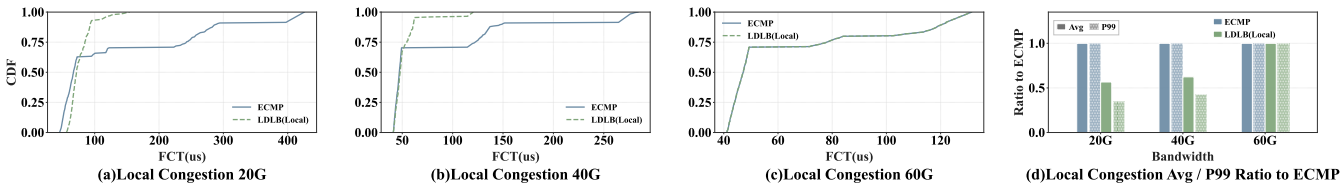


Fig. 7. (a), (b), (c) CDF of ECMP and LDLB (Local) under different congested link bandwidths, and (d) the average FCT of the two algorithms.

scale and connectivity as Scenario 1, but with different link-rate settings: the inter-switch core links are set to 100 Gbps, and the host NIC bandwidth is set to 25 Gbps. **Topology 3** uses a three-tier Fat-Tree topology with 16 core switches, 20 aggregation switches, and 20 edge switches. All inter-switch links are configured at 400 Gbps. Hosts are connected to edge switches, and each edge-to-host access link operates at 100 Gbps.

Traffic Workload: To evaluate LDLB under different congestion conditions, we designed two traffic scenarios.

The first is a persistent-congestion scenario. We inject long-lived background flows and reduce the bandwidth of selected links to emulate (i) local congestion (leaf egress congestion), (ii) global congestion (spine egress congestion), and (iii) hybrid local-global congestion. This asymmetric configuration, where only selected links are throttled while all others remain at full capacity, intentionally creates controlled and reproducible congestion points that emulate the scenario where long-lived flows persistently occupy specific links — precisely the worst case that LDLB is designed to address. The bandwidth is gradually reduced from 60 Gbps to 20 Gbps to systematically evaluate LDLB's congestion detection and rerouting effectiveness under varying congestion intensities while isolating the impact of individual congested links. Hosts simultaneously generate short flows as target traffic for performance evaluation. Under the hybrid local-global setting, we further generate target traffic using four representative real-world workloads: Web Server (WSv), Cache Follower (CF), Web Search (WSc), and Data Mining (DM). **The second** is a bursty-congestion scenario, where we use a traffic pattern of "four real-world workloads + 500 KB burst aggregation" to emulate transient congestion events. Unless otherwise specified, traffic load is controlled by adjusting the flow arrival rate; flow arrivals follow a Poisson process. Source-destination host pairs are selected at random. All compared algorithms are evaluated under identical traffic conditions to ensure fairness.

Evaluation Metrics: FCT and its cumulative distribution function (CDF) are core metrics in our experimental evaluation. FCT measures the end-to-end latency of an individual flow from initiation to completion, while the CDF characterizes the overall distribution of FCT, capturing its central tendency and dispersion, and revealing tail-latency behavior.

LDLB is compared with six representative load balance algorithms (ECMP, DRILL, LetFlow, CONGA, HULA and ConWeave) across distinct congestion scenarios.

B. *Performance Under Persistent Congestion Scenarios*

*1) **Local Congestion Scenario**:* To evaluate the performance of LDLB (Local) under persistent local congestion, the experiment first injects multiple long flows as background traffic to continuously occupy selected links. Short flows are then generated within the terminal cluster associated with the long flows as foreground traffic for evaluation. Meanwhile, the bandwidth of the leaf-switch egress links traversed by long flows is gradually reduced from 60 Gbps to 20 Gbps, while all other links remain fixed at 400 Gbps. This setup creates sustained congestion with different intensities on multiple leaf egress links, thereby emulating a local-congestion environment in data center networks. In addition, the experiment employs RoCEv2, which is based on UDP, as the transport protocol. To ensure reliable transmission can still be maintained under congested conditions, sufficient buffers are configured at the senders, receivers, and switches to minimize the impact of packet loss and retransmissions caused by congestion on transmission performance.

As shown in Fig. 7(a)–(d), we compare LDLB (Local) and ECMP across three congestion levels. The congested link bandwidth is 20 Gbps in (a), 40 Gbps in (b), and 60 Gbps in (c). At 20 Gbps (Fig. 7a), ECMP's tail extends substantially while LDLB maintains a compact distribution. The root cause is that ECMP's static hash continues assigning traffic to the bottleneck port with the same probability as to healthy ports, causing compounding queue buildup. LDLB breaks this cycle by detecting the congested queue via RED/ECN and removing the port from the candidate set. At 40 Gbps (Fig. 7b), ECMP's CDF shifts rightward with a visible tail, while LDLB (Local) reduces average FCT by 38% by excluding congested ports. A distinctive knee appears in LDLB's CDF near the tail—this corresponds to flows that, after the congested port is removed, are hashed onto remaining ports that still exhibit residual imbalance from ECMP's hash distribution. At 60 Gbps (Fig. 7c), the CDF curves nearly overlap, confirming that LDLB's congestion-detection overhead is negligible under light load.

*2) **Global Congestion Scenario**:* Since LDLB (Local) lacks global congestion awareness, we further develop a global label-based routing mechanism and systematically evaluate the effectiveness of LDLB (Global) under global congestion scenarios. Specifically, multiple long flows are first injected as background traffic to continuously occupy selected links, after which short flows are introduced as foreground traffic. Meanwhile, the bandwidth of spine-switch egress links traversed by long flows is gradually reduced from 60 Gbps to 20 Gbps, while all remaining links are fixed at 400 Gbps. This setup creates different levels of congestion on spine egress links, thereby emulating a global congestion environment in data center networks.

As shown in Fig. 8(a)–(d), we compare six schemes under

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

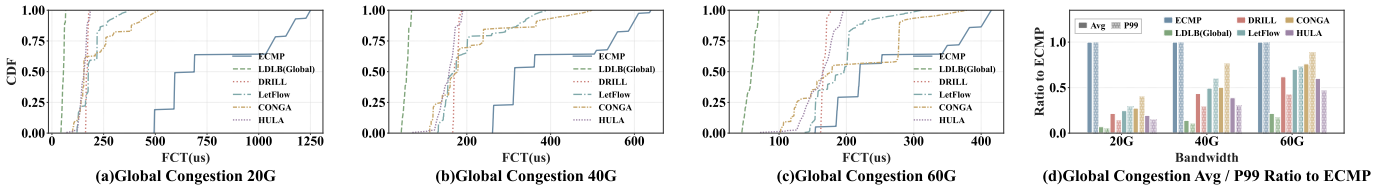


Fig. 8. (a), (b), (c) (CDF of ECMP, LDLB(Global), DRILL, LetFlow, HULA and CONGA under different congested link bandwidth conditions, and (d) shows the average FCT ratio of the four algorithms compared to ECMP.

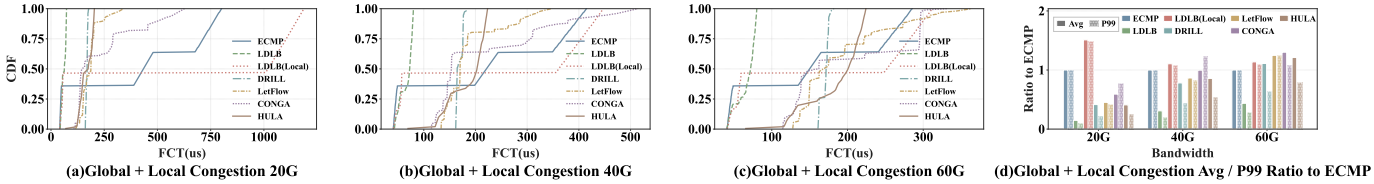


Fig. 9. (a), (b), (c) CDF of ECMP, LDLB, LDLB (Local), DRILL, LetFlow, HULA and CONGA under different congested link bandwidth conditions, and (d) presents the average FCT ratio of the five algorithms compared to ECMP.

this purely downstream congestion scenario—the critical test for global synchronization effectiveness. At 20 Gbps (Fig. 8a), ECMP's CDF tail is longest, as its hash-blind forwarding cannot avoid the spine bottleneck despite the local switch being congestion-free. DRILL and LetFlow also degrade significantly: their local-only mechanisms see no queue buildup on leaf ports and therefore cannot detect the downstream spine congestion. CONGA and HULA, though globally aware, suffer from feedback latency—CONGA's reverse-path piggybacking requires at least one RTT, and HULA's probe polling introduces interval delay—so their reaction lags behind the congestion evolution. LDLB (Global) achieves the most compact CDF and lowest average FCT because its proactive label mechanism synchronizes the downstream congestion state to upstream switches in a single hop, enabling immediate rerouting at the leaf before traffic enters the congested spine link. At 40 Gbps (Fig. 8b), all CDFs shift left and gaps narrow. At 60 Gbps (Fig. 8c), all algorithms converge. The summary (Fig. 8d) confirms that LDLB (Global)'s gain is most significant under severe congestion and diminishes as bandwidth increases.

*3) **Local + Global Congestion Scenario**:* In real-world data centers, network topology and traffic dynamics are highly complex and time-varying. To better approximate practical deployments, we construct a composite scenario that includes both local and global congestion, and introduce heterogeneous link capacities to create non-equivalent multi-hop paths. Specifically, multiple long flows are first injected as background traffic to continuously occupy selected links, after which short flows are introduced as foreground traffic. Meanwhile, the bandwidth of spine and leaf egress links traversed by long flows is gradually reduced from 60 Gbps to 20 Gbps, while other links remain unchanged. This setup induces different congestion intensities on multiple spine and leaf egress links, thereby forming a joint local-and-global congestion environment.

As shown in Fig. 9(a)–(d), the result trends are consistent across all subfigures: a critical finding is that LDLB (Local) performs over 10% worse than ECMP at 20 Gbps. The root cause is that purely local awareness cannot perceive downstream spine congestion—it sees no queue buildup on its own leaf egress ports and continues forwarding traffic into the bottlenecked spine link. ECMP, though congestion-unaware, occasionally outperforms LDLB (Local) purely by hash randomness. LDLB (Global+Local) resolves this fundamental limitation by combining local RED/ECN detection for leaf congestion with globally propagated labels for spine congestion, improving FCT by over 50% versus ECMP and over 2× other baselines with the most compact CDF across all subfigures.

The consistent ranking across bandwidth levels reveals how each scheme's mechanism determines its performance under combined congestion: CONGA and HULA, despite possessing global awareness, suffer from feedback delays that limit their responsiveness; DRILL and LetFlow, being local-only, are simultaneously blind to both leaf and spine congestion. At 60 Gbps, several schemes show FCT degradation relative to ECMP because their continuous control overhead (queue sampling, flowlet tracking, probes) exceeds congestion-avoidance benefits under light load, while LDLB generates zero control traffic on uncongested ports.

Overall, these results demonstrate that LDLB provides significant advantages in complex scenarios with non-equivalent links and simultaneous local-global congestion. It more effectively utilizes residual bandwidth, thereby reducing overall FCT while improving network throughput.

*4) **Real Workload Scenarios With Persistent Congestion**:* To make the evaluation results more convincing, this paper further validates using four real-world workloads widely applied in production data centers, including Web Server (WSv), Cache Follower (CF), Web Search (WSc), and Data Mining (DM), whose traffic exhibits a heavy-tailed distribution. The average flow sizes of these workloads are 64KB, 701KB, 1.6MB, and 7.41MB, respectively. Specifically, approximately 81%, 53%, 62%, and 83% of the traffic is less than 100KB, while the proportions of traffic greater than 1MB are 0%, 29%, 20%, and 9%, respectively. This paper uses flows generated from four general datasets as target flows and inputs them into

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

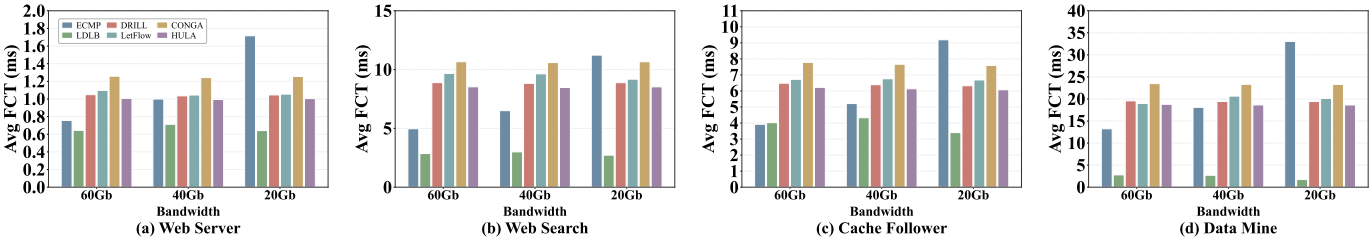


Fig. 10. The average FCT for the Web Server, Cache Follower, Web Search, and Data Mining workloads in local and global congestion scenarios.

the topology model with local+global congestion.

As shown in Fig. 10(a)-(d), a clear pattern emerges across all four workloads. At 60 Gbps and 40 Gbps, DRILL, LetFlow, HULA, and CONGA all exhibit higher average FCT than ECMP—their control overhead yields insufficient congestion-avoidance benefit under light congestion. As the bottleneck tightens to 20 Gbps, these schemes transition from degradation to improvement over ECMP, as congestion avoidance finally outweighs overhead. LDLB achieves the lowest FCT at all bandwidth levels without any degradation regime.

Notably, HULA and CONGA perform consistently worse than DRILL and LetFlow despite having global awareness. The root cause is twofold. First, HULA and CONGA incur continuous control traffic (probes, per-flowlet feedback) that consumes bandwidth on congested links, while DRILL and LetFlow add negligible overhead. Second, in this persistent fixed-bottleneck scenario, global feedback staleness means congestion information often arrives too late to affect in-flight traffic, whereas DRILL's local sampling and LetFlow's flowlet randomization react immediately at the switch. The overhead of global coordination thus yields no proportional benefit under sustained fixed-point congestion.

LDLB avoids both pitfalls. Its label mechanism is triggered only upon congestion detection, generating zero control traffic on uncongested ports—eliminating the degradation at 60/40 Gbps. Its proactive one-hop label notification delivers congestion signals in sub-RTT without waiting for reverse traffic or probe cycles. Combined with simultaneous local and global awareness, LDLB covers both leaf and spine bottlenecks, unlike local-only or delayed-global schemes.

This paper conducts evaluations using the UDP-based RoCEv2 transport protocol. Although RoCEv2 provides reliable transmission guarantees at the link layer, its performance is highly dependent on lossless network conditions. Under the congestion scenarios constructed in the experiments, even with lossless mechanisms such as PFC configured, high-intensity background traffic may still cause transient exhaustion of switch buffers or trigger PFC backpressure, leading to micro-burst packet loss or latency. This effect is amplified in throughput-sensitive flows such as Web Search and Data Mining, resulting in worse average FCT compared to Cache Follower, which is more latency-sensitive and exhibits different traffic characteristics. If TCP were used in the same network topology, its end-to-end congestion control and packet loss retransmission mechanisms, while ensuring data integrity, may further increase the average FCT for these throughput-sensitive workloads due to higher latency and protocol overhead. Therefore, in RDMA networks, the co-design of the transport protocol, underlying lossless network mechanisms, and load-balance algorithms is crucial for guaranteeing both high throughput and data integrity.

C. *Performance Under Burst Congestion Scenarios*

*1) **Test of Congestion Label Compression**:* We implement a lightweight congestion-label compression mechanism (CLC) in ns-3 by aggregating labels with the same IP prefix into a single entry and retaining only the minimum and maximum values for contiguous suffix ranges. This design reduces label overhead while preserving essential congestion semantics. To evaluate both the effectiveness of CLC and the coupling behavior between LDLB and DCTCP, we build a data center network based on Topology 2 and conduct comparative experiments on four representative workloads. We compare LDLB, LDLB/CLC, LDLB+DCTCP, and LDLB/CLC+DCTCP under two load levels (0.3 and 0.9). Flow arrivals follow a Poisson process, host pairs are randomly matched, and an additional 500 KB, 50:1 incast pattern is superimposed to emulate bursty many-to-one congestion.

As shown in Fig. 11 and Fig. 12, ECMP performs worst overall across workloads and load levels, indicating that static hash-based forwarding is insufficient for incast-induced transient hotspots. Compared with ECMP, LDLB is more robust under different configurations (with/without DCTCP and with/without CLC), achieving stable gains in both Avg FCT and P99 FCT, with peak improvements up to 90%. After enabling CLC, LDLB shows no degradation relative to the uncompressed baseline and further achieves an additional 0.2%–44% FCT improvement (under the assumption that hosts under the same leaf share the same prefix and contiguous suffixes). Although CLC introduces modest computation overhead, it substantially reduces control-plane label traffic, improves label lookup and congestion-table update efficiency, and lowers switch memory usage.

To validate the above findings, we further analyze control-plane metrics under different workloads and load levels in Fig. 13 and Fig. 14, including control-plane traffic, its ratio to total traffic, and congestion convergence time—defined as the interval from congestion detection to successful synchronization of congestion notifications. The experimental results demonstrate that, compared to LDLB, LDLB+CLC reduces congestion-label traffic by at least 30%, achieving the lowest control-plane overhead among all schemes. Its congestion synchronization time is also shortened by at least 20%, approaching the performance of ConWeave. The low control

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

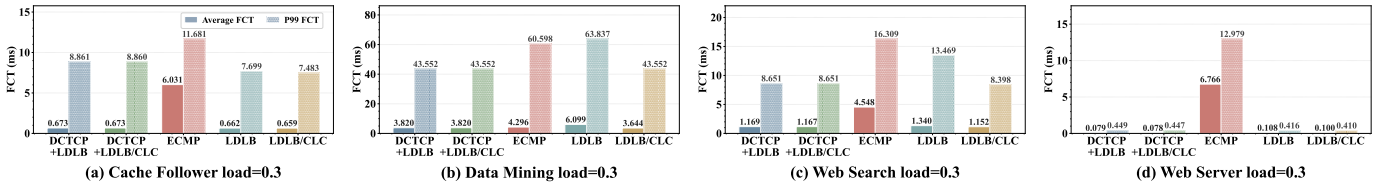


Fig. 11. Performance comparison of different algorithms under load 0.3 with the extended mechanism configuration, showing the variation trends of Avg FCT and P99 FCT across workloads.

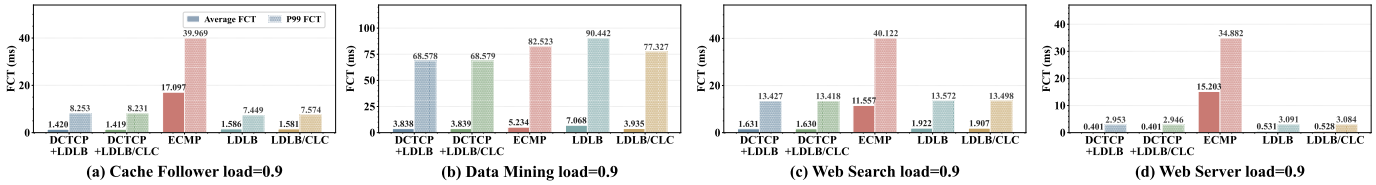


Fig. 12. Performance comparison of different algorithms under load 0.9 with the extended mechanism configuration, showing the robustness of Avg FCT and P99 FCT under high load and burst-congestion scenarios.

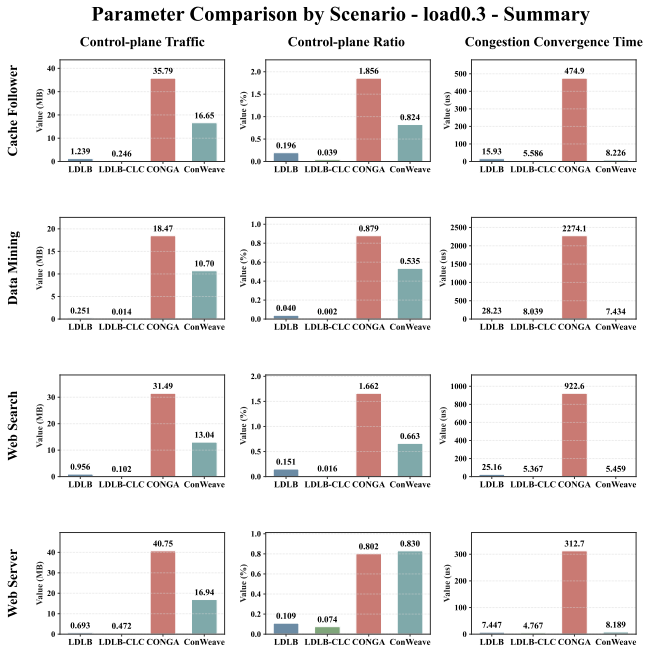


Fig. 13. Control-plane overhead comparison under the 0.3 incast workload. The figure shows control traffic, control-to-data traffic ratio, and congestion convergence time across LDLB, LDLB/CLC, CONGA, and ConWeave for cache follower, data mining, web search, and web server scenarios.

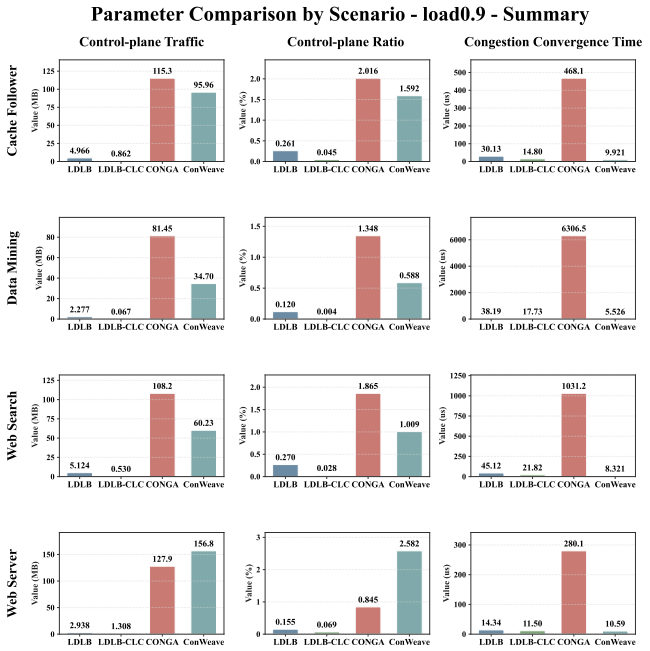


Fig. 14. Control-plane overhead comparison under the 0.9 incast workload. The figure shows control traffic, control-to-data traffic ratio, and congestion convergence time across LDLB, LDLB/CLC, CONGA, and ConWeave for cache follower, data mining, web search, and web server scenarios.

overhead of LDLB stems from its mechanism of generating notifications only after congestion is triggered and filtering duplicate congestion information. In contrast, CONGA relies on continuous flowlet-level feedback, and downstream congestion information must be carried back via reverse flowlets, making its synchronization latency more vulnerable to in-network congestion. The overhead of ConWeave mainly comes from per-flow, RTT-level proactive feedback packets (with the same priority as ACKs), resulting in a relatively higher control-plane share but shorter synchronization latency. The synchronization delay of LDLB is slightly longer than that of ConWeave, primarily due to the overhead introduced by ECN-based decisions and CLC preprocessing.

In addition, Fig. 11 and Fig. 12 reveal two notable features. First, native LDLB (without DCTCP) exhibits higher average FCT than ECMP in the Data Mining workload, but not in other workloads. The root cause is that DM's multiple long flows (∼50 MB) contend for the same last-hop link (the leaf-to-host egress port). This is a special case where the congested port is the final hop to the destination—LDLB cannot simply exclude it, as doing so leaves no forwarding path. Since the congestion is at the last hop beyond the switch's routing control, LDLB's port-exclusion mechanism is inherently ineffective here, and ECMP faces the same limitation as it cannot avoid a single last-hop bottleneck either. Enabling DCTCP resolves this degradation: DCTCP's end-host rate control reduces the sending rate of long flows, alleviating last-hop queue buildup. Adding CLC further improves performance

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

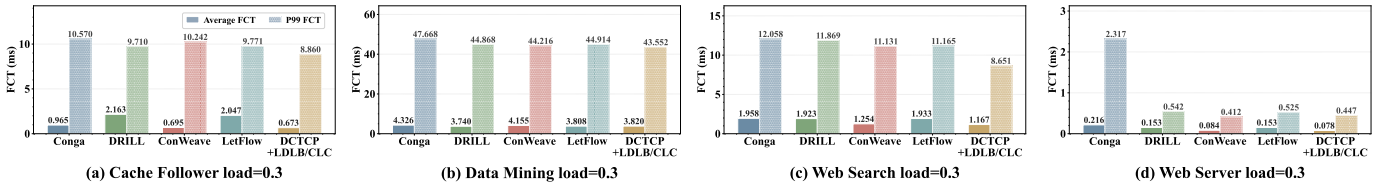


Fig. 15. Comparison of Avg FCT and P99 FCT among different load-balance algorithms under load 0.3 across four data-center workloads.

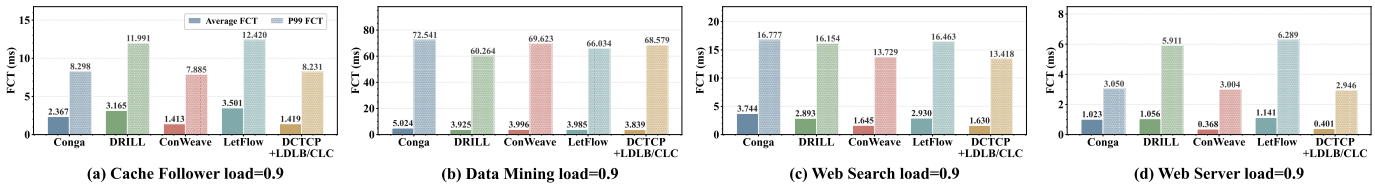


Fig. 16. Comparison of Avg FCT and P99 FCT among different load-balance algorithms under load 0.9 across four data-center workloads, highlighting differences in congestion-handling capability under high load.

under heavy load by reducing label traffic volume, which frees bandwidth for data packets on already saturated links.

Second, under load 0.3, LDLB+DCTCP shows 2–4% higher average latency than LDLB alone, due to control-coupling mismatch: after LDLB has already rerouted traffic away from congested links, transient ECN marks generated by micro-bursts still trigger unnecessary end-host rate reduction via DCTCP, causing a temporary overreaction. Under high load (0.9), this effect disappears because congestion is sustained rather than transient, and both mechanisms work synergistically, yielding 9–45% improvement.

*2) **Real Workload With Incast**:* To systematically characterize performance differences across workload types and load levels, we conduct comparative experiments on Topology 2 against CONGA, DRILL, LetFlow, and ConWeave under four representative data-center workloads. Two load settings (0.3 and 0.9) are considered. Traffic arrivals follow a Poisson process, host pairs are randomly matched, and an additional 500 KB, 50:1 incast pattern is superimposed to emulate bursty many-to-one congestion.

As shown in Fig. 15 to Fig. 16, LDLB achieves about 30%–60% performance gains over CONGA, DRILL, and LetFlow in most scenarios. Compared with the newer congestion-aware scheme ConWeave, LDLB is overall comparable and further outperforms it in several workload/load combinations. It is worth noting that the ranking of algorithms is not identical across all workloads; however, LDLB and ConWeave consistently remain in the first tier. These trends can be explained by differences among the five schemes in congestion-awareness capability, decision granularity, and rerouting execution mechanisms.

CONGA performs path-aware load balance with flowlet-level rerouting at the source ToR. Although it improves over static hashing, its effectiveness depends on flowlet boundaries; in long-flow-dominant scenarios, insufficient flowlet gaps limit rerouting opportunities, and reverse-path-dependent feedback is sensitive to network dynamics, resulting in slower convergence and less stable tail latency under persistent hotspots. DRILL adopts a lightweight queue-aware local decision strategy with low overhead, but limited global congestion visibility can lead to local-optimum/global-suboptimum behavior under burst contention. LetFlow relies on natural flowlet gaps and randomized spreading; while scalable, it provides limited proactive suppression of synchronized hotspots in high-load incast settings, and thus often underperforms in both Avg FCT and P99 FCT. ConWeave combines fine-grained congestion feedback with fast rerouting and in-network reordering, typically delivering faster convergence and stronger tail-latency control; however, in long-flow-heavy workloads (e.g., Data Mining), where congestion is persistent rather than transient, the marginal gain of aggressive dynamic rerouting decreases and control/coordination overhead becomes more pronounced. LDLB performs path avoidance using explicit congestion labels and can be coupled with CLC and DCTCP: CLC reduces control-plane overhead and accelerates notification dissemination/table updates, while DCTCP provides end-host rate regulation under sustained congestion. Consequently, LDLB remains in the same first tier as ConWeave and is often superior when stable congestion avoidance and low control-plane overhead are both required.

In summary, algorithm ranking is jointly determined by three capabilities: fast and accurate congestion awareness, robust congestion decision and path avoidance, and controllable rerouting side effects. This also explains workload-dependent ranking shifts: under short-flow burst-dominant conditions, ConWeave and LDLB are usually strongest; under long-flow persistent-congestion conditions, LDLB often shows better robustness.

*3) **FAT-Tree Topology Tests**:* To validate the generality of LDLB and its compression extension (CLC) in complex topologies, this paper compares LDLB, LDLB/CLC, and ECMP on a Fat-Tree topology under a load of 0.9. The evaluation is conducted on four typical workloads—Cache Follower, Data Mining, Web Search, and Web Server—measuring both average and tail (P99) flow completion time. The results in Fig. 17 show that LDLB achieves consistent improvements over ECMP in average latency, ranging from 8.9% to 62.0%, with the most significant gain observed in the Web Server scenario. However, the improvement in tail latency varies across workloads: Web Server and Web Search show improvements

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

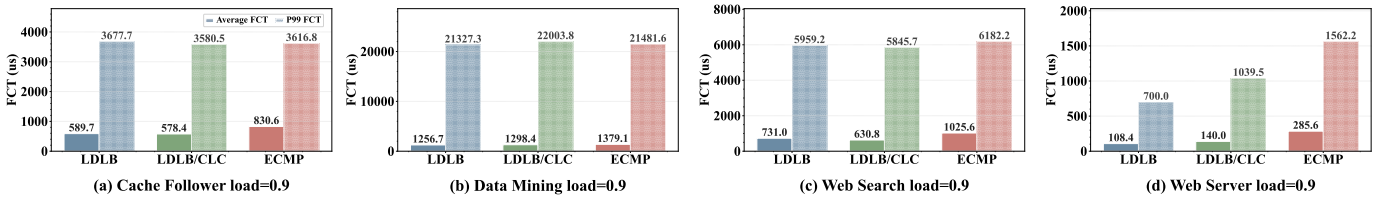


Fig. 17. Comparison of Avg FCT and P99 FCT among different load-balance algorithms under load 0.9 across four data-center workloads, highlighting differences in congestion-handling capability under high load.

of 55% and 3%, respectively, while Data Mining exhibits only a marginal gain of 0.7%, and Cache Follower shows a slight degradation of 1.7%. After introducing CLC, further improvements in average latency are observed for Cache Follower and Web Search (30% and 38% over ECMP, respectively), whereas for Data Mining and Web Server, LDLB/CLC slightly underperforms the original LDLB. This indicates that the benefits of CLC are highly workload-dependent.

The performance differences stem from the interplay between workload traffic patterns and topological characteristics. First, the higher path symmetry and richer alternative paths in Fat-Tree enhance the baseline performance of ECMP, thereby narrowing the relative improvement space for LDLB. Second, congestion in Fat-Tree tends to be distributed across multiple layers, making localized congestion avoidance less effective in improving tail latency compared to Leaf-Spine topologies. For CLC, its effectiveness essentially represents a trade-off between the benefit of reduced control-plane overhead and the cost of compression/decompression and propagation latency. CLC is more likely to yield positive gains when label redundancy is high and compression does not significantly impact timeliness; conversely, in scenarios sensitive to control latency or with persistent congestion, the additional processing overhead may offset part of the gains. Specifically, in Cache Follower and Web Search, where label redundancy is relatively high, CLC effectively reduces control overhead through signal compression, and its benefits outweigh the processing delay cost. In Data Mining, persistent large flows lead to network bandwidth saturation, and the slight processing delay introduced by CLC prevents it from achieving a net gain in such bandwidth-constrained environments. In Web Server, which is dominated by latency-sensitive short flows with transient micro-burst congestion patterns, the aggregation and decompression delays introduced by CLC, though small, cause scheduling decisions to lag behind rapidly changing congestion states. As a result, some short flows may be misdirected to congested links before optimal path information takes effect, thereby undermining tail latency optimization.

In summary, while LDLB still generally outperforms ECMP in Fat-Tree, the extent of its advantage and the additional gains from CLC are significantly modulated by workload traffic patterns and network congestion distribution. This reveals that in topologies with high path redundancy, dynamic load balance mechanisms require a careful trade-off between control-plane timeliness and overhead. Future work may explore workload-adaptive strategies to further enhance the generality of the mechanism.

V. Related Work

In recent years, a variety of congestion control mechanisms have been proposed to address diverse network scenarios. QCN[36] quantifies congestion at the link layer for Ethernet, while DCQCN[37] combines ECN marking with PFC suppression for RDMA. TIMELY[38] and Swift[39] use RTT as a congestion signal for delay-aware rate control. MP-RDMA[40] proposes a multi-path acknowledgment clock scheme for RDMA. HPCC[33] employs INT-based telemetry for per-hop congestion detection and precise rate adjustment. PCN[41] regulates only identified congested flows rather than all traffic, and TCD[42] detects congestion in ongoing transmissions. BFC[43] applies per-flow, hop-by-hop backpressure to eliminate Head-of-Line blocking, while Dart[44] decomposes congestion relief into receiver-side and in-network sub-tasks.

Since LDLB relies on RED/ECN for local congestion detection, the accuracy of ECN marking directly affects its routing decisions. In multi-queue switch architectures, per-port ECN marking may cause non-congested queues to be erroneously marked due to buffer pressure from other queues sharing the same output port. ECN+[54] proposes a marking-aware optimization that excludes already-marked packets from the output port buffer's marking decision, thereby preventing fault marking. DC-ECN[52] leverages Gaussian Process Regression to separate mice and elephant flows into dual couple buffers, each with independently tuned dynamic ECN thresholds (K_{min} and K_{max}). Priority-ECN[55] combines the marking-aware strategy with priority scheduling for marked packets and a Cut Payload mechanism, ensuring that congestion signals reach the sender early for timely rate adjustment. These techniques can complement LDLB by improving its local congestion detection accuracy in multi-queue environments.

LDLB can further benefit from DRILL's[32] "power-of-two-choices" principle for route updates and HPCC's[33] telemetry-based congestion assessment. It can also synergize with centralized schemes[12], [13] or centralized controllers[14], [15], and incorporate dynamically tunable ECN thresholds[50], [51], [53].

VI. Conclusion

In this paper, we proposed LDLB for data center networks. LDLB bridges the gap between local and global load balance by combining instantaneous local congestion detection (via RED/ECN and the LCT) with globally synchronized congestion labels (stored in the CLT). A compressible information table reduces label storage and synchronization overhead, and a WCMP-like cost function enables congestion-aware

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

rerouting. Stability is ensured through a congestion-delay timer and flow-pinning mechanism.

Experiments on ns-3 with four real data center workloads demonstrate that LDLB reduces average FCT by 57%, 61%, 65%, 67%, and 54% compared with ECMP, DRILL, LetFlow, HULA, and CONGA, respectively. LDLB also performs comparably to ConWeave across workloads, with lower control-plane overhead. The CLC extension reduces congestion-label traffic by at least 30% and convergence time by at least 20%. These results hold across both Leaf-Spine and Fat-Tree topologies under persistent and bursty congestion scenarios.

Since traditional switches do not support floating-point operations, the cost-function computation can be implemented using FPGA-based switches[34] or fixed-point arithmetic on programmable ASICs. Future work includes hardware prototyping on P4-programmable switches[35], workload-adaptive weight tuning, and integration with INT-based telemetry for finer-grained congestion assessment.

REFERENCES

[1] K. Qian et al., "Alibaba HPN: a data center network for large language model training," in Proc. ACM SIGCOMM, 2024, pp. 691–706.

[2] Y. Chang et al., "A survey on evaluation of large language models," ACM Trans. Intelligent Systems and Technology, vol. 15, no. 3, 2024, pp. 1–45.

[3] A. VasanthaAzhagu et al., "Cloud computing overview, security threats and solutions-a survey," in Proc. ACM ICIA, 2016, pp. 1–6.

[4] M. Noormohammadpour et al., "Datacenter traffic control: Understanding techniques and tradeoffs," IEEE Commun. Surveys Tuts., vol. 20, no. 2, 2017, pp. 1492–1525.

[5] Z. Zhang et al., "Hashing linearity enables relative path control in data centers," in Proc. USENIX ATC, 2021, pp. 855–862.

[6] C. Hopps, "Analysis of an equal-cost multi-path algorithm," RFC 2992, 2000.

[7] A. Dixit et al., "On the impact of packet spraying in data center networks," in Proc. IEEE INFOCOM, 2013, pp. 2130–2138.

[8] J. Zhou et al., "WCMP: Weighted cost multipathing for improved fairness in data centers," in Proc. EuroSys, 2014, pp. 1–14.

[9] S. Ghorbani et al., "Drill: Micro load balance for low-latency data center networks," in Proc. ACM SIGCOMM, 2017, pp. 225–238.

[10] E. Vanini et al., "Let it flow: Resilient asymmetric load balance with flowlet switching," in Proc. USENIX NSDI, 2017, pp. 407–420.

[11] M. Alizadeh et al., "CONGA: Distributed congestion-aware load balance for datacenters," in Proc. ACM SIGCOMM, 2014, pp. 503–514.

[12] K. He et al., "Presto: Edge-based load balance for fast datacenter networks," in Proc. ACM SIGCOMM, 2015, pp. 465–478.

[13] A. Kabbani et al., "Flowbender: Flow-level adaptive routing for improved latency and throughput in datacenter networks," in Proc. ACM CoNEXT, 2014, pp. 149–160.

[14] M. Al-Fares et al., "Hedera: Dynamic flow scheduling for data center networks," in Proc. USENIX NSDI, 2010, pp. 89–92.

[15] J. Perry et al., "Fastpass: A centralized zero-queue datacenter network," in Proc. ACM SIGCOMM, 2014, pp. 307–318.

[16] X. Tao et al., "Key flow first prioritized flow scheduling strategy in multi-tenant data centers," IEEE Trans. Network and Service Management, 2024.

[17] M. Zhao et al., "A survey of data center network topology structure," in Proc. IEEE ICACT, 2023, pp. 303–309.

[18] B. Lebiednik et al., "A survey and evaluation of data center network topologies," arXiv preprint arXiv:1605.01701, 2016.

[19] C. Guo et al., "Pingmesh: A large-scale system for data center network latency measurement and analysis," in Proc. ACM SIGCOMM, 2015, pp. 139–152.

[20] Q. Fan et al., "Towards workload balance in fog computing empowered IoT," IEEE Trans. Network Science and Engineering, vol. 7, no. 1, 2018, pp. 253–262.

[21] S. Floyd et al., "Random early detection gateways for congestion avoidance," IEEE/ACM Trans. Networking, 1996, pp. 375–385.

[22] M. Alizadeh et al., "Data center TCP (DCTCP)," in Proc. ACM SIGCOMM, 2010, pp. 63–74.

[23] H. Wu et al., "Tuning ECN for data center networks," in Proc. ACM CoNEXT, 2012, pp. 25–36.

[24] W. Bai et al., "Enabling ECN in multi-service multi-queue data centers," in Proc. USENIX NSDI, 2016, pp. 537–549.

[25] F. Baker et al., "A differentiated services code point (DSCP) for capacity-admitted traffic," RFC 5865, 2010.

[26] T. Benson et al., "Understanding data center traffic characteristics," in Proc. ACM SIGCOMM, 2010, pp. 92–99.

[27] J. Hu et al., "RPO: Receiver-driven transport protocol using opportunistic transmission in data center," in Proc. IEEE ICNP, 2021, pp. 1–11.

[28] Z. Li et al., "Rate-aware flow scheduling for commodity data center networks," in Proc. IEEE INFOCOM, 2017, pp. 1–9.

[29] Y. Zhao et al., "Joint VM placement and topology optimization for traffic scalability in dynamic datacenter networks," Computer Networks, 2015, pp. 109–123.

[30] C. Hu et al., "Discount counting for fast flow statistics on flow size and flow volume," IEEE/ACM Trans. Networking, vol. 22, no. 3, 2013, pp. 970–981.

[31] W. Bai et al., "Information agnostic flow scheduling for commodity data centers," in Proc. USENIX NSDI, 2015, pp. 455–468.

[32] M. Mitzenmacher, "The power of two choices in randomized load balance," IEEE Trans. Parallel and Distributed Systems, vol. 12, no. 10, 2001, pp. 1094–1104.

[33] Y. Li et al., "HPCC: High precision congestion control," in Proc. ACM SIGCOMM, 2019, pp. 44–58.

[34] P. Papaphilippou et al., "High-performance FPGA network switch architecture," in Proc. ACM/SIGDA FPGA, 2020, pp. 76–85.

[35] S. M. Shahbaz et al., "Pisces: A programmable, protocol-independent software switch," in Proc. ACM SIGCOMM, 2016, pp. 525–538.

[36] IEEE 802.1Qau, "Congestion Notification," 2024. [Online]. Available: http://www.ieee802.org/1/pages/802.1au.html

[37] Y. Zhu et al., "Congestion control for large-scale RDMA deployments," in Proc. ACM SIGCOMM, 2015, pp. 523–536.

[38] R. Mittal, "Timely: RTT-based congestion control for the datacenter," in Proc. ACM SIGCOMM, 2015, pp. 537–550.

[39] G. Kumar et al., "Swift: Delay is simple and effective for congestion control in the datacenter," in Proc. ACM SIGCOMM, 2020, pp. 514–528.

[40] Y. Lu et al., "Multi-path transport for RDMA in datacenters," in Proc. USENIX NSDI, 2018, pp. 357–371.

[41] W. Cheng et al., "Re-architecting congestion management in lossless Ethernet," in Proc. USENIX NSDI, 2020, pp. 19–36.

[42] Y. Zhang et al., "Congestion detection in lossless networks," in Proc. ACM SIGCOMM, 2021, pp. 370–383.

[43] P. Goyal et al., "Backpressure flow control," in Proc. USENIX NSDI, 2022, pp. 1–3.

[44] J. Xue et al., "DART: Divide and specialize for fast response to congestion in RDMA-based datacenter networks," IEEE/ACM Trans. Networking, vol. 28, no. 1, 2020, pp. 322–335.

[45] N. Katta et al., "HULA: Scalable load balance using programmable data planes," in Proc. SOSR, 2016, pp. 1–12.

[46] H. Zhang et al., "Resilient datacenter load balance in the wild," in Proc. ACM SIGCOMM, 2017, pp. 253–266.

[47] N. Katta et al., "Clove: Congestion-aware load balance at the virtual edge," in Proc. ACM CoNEXT, 2017, pp. 323–335.

[48] J. Hu et al., "Adjusting switching granularity of load balance for heterogeneous datacenter traffic," IEEE/ACM Trans. Networking, vol. 29, no. 5, 2021, pp. 2367–2384.

[49] T. Benson et al., "MicroTE: Fine grained traffic engineering for data centers," in Proc. ACM CoNEXT, 2011, pp. 1–12.

[50] S. Wang et al., "A-ECN: Minimizing queue length for datacenter networks," IEEE Access, vol. 8, pp. 49100–49111, 2020.

[51] T. Zhang et al., "Adaptive marking threshold method for delay-sensitive TCP in data center network," J. Network and Computer Applications, vol. 61, pp. 222–234, 2015.

[52] A. Majidi et al., "DC-ECN: A machine-learning based dynamic threshold control scheme for ECN marking in DCN," Computer Communications, vol. 150, pp. 334–345, 2019.

[53] Y. Lu et al., "Dynamic ECN marking threshold algorithm for TCP congestion control in data center networks," Computer Communications, vol. 129, pp. 197–208, 2018.

[54] A. Majidi et al., "ECN+: A marking-aware optimization for ECN threshold via per-port in data center networks," J. Network and Computer Applications, vol. 152, p. 102504, 2019.

[55] A. Majidi et al., "Priority policy in multi-queue data center networks via per-port ECN marking," in Proc. Int. Conf. Ubiquitous Information Management and Communication (IMCOM), 2020, pp. 1–8.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Cloud Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TCC.2026.3723429

[56] C. Tan et al., "NetBouncer: Active device and link failure localization in data center networks," in Proc. USENIX NSDI, 2019, pp. 599–614.
[57] H. Herodotou et al., "Scalable near real-time failure localization of data center networks," in Proc. ACM SIGKDD, 2014, pp. 1689–1698.
[58] P. Gill et al., "Understanding network failures in data centers: Measurement, analysis, and implications," in Proc. ACM SIGCOMM, 2011, pp. 350–361.
[59] C. H. Song et al., "Network load balancing with in-network reordering support for RDMA," in Proc. ACM SIGCOMM, 2023, pp. 816–831.
[60] M. A. Qureshi et al., "PLB: Congestion signals are simple and effective for network load balancing," in Proc. ACM SIGCOMM, 2022, pp. 207–218.
[61] snowzjx, "ns-3 load balance simulation framework," 2019. [Online]. Available: https://github.com/snowzjx/ns3-load-balance
[62] Alibaba Education, "High Precision Congestion Control," 2021. [Online]. Available: https://github.com/alibaba-edu/High-Precision-Congestion-Control
[63] ConWeave Project, "ConWeave ns-3 simulation," 2024. [Online]. Available: https://github.com/conweave-project/conweave-ns3



Xiaoyan Qian Xiaoyan Qian received the B.S. and Ph.D. degrees in computer science from Nanjing University of Science and Technology, China. She is currently an Associate Professor with Nanjing University of Aeronautics and Astronautics. She has published more than 40 journal articles and one book. Her current research interests include deep learning and intelligent surveillance.



Rui Xu Rui Xu received the B.E. degree from the College of Civil Aviation at Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2024. He is currently pursuing a master's degree as a graduate student at the same university. His research interests include data center network and traffic engineering.



Lei Han Lei Han is currently a Professor at the School of Computer, Nanjing University of Posts and Telecommunications. He has created patents authorized by China, the US, the UK, Japan, Korea, and Germany. He has authored papers in international conferences and journals such as IEEE Transactions on Parallel and Distributed Systems, GLOBECOM, MOBISYS, and ICCN. His research interests include computer networks and datacenter systems.



Xin He Xin He (Member, IEEE) received the M.E. degree from the School of Automation at Beijing Institute of Technology in 2016 and the Ph.D. degree from the Department of Computer Science and Technology at Nanjing University in 2021. He is currently a lecturer with the School of Computer Science, Nanjing University of Posts and Telecommunications. His research interests include edge computing, SDN, and data center networks.



Shuwen Wei Shuwen Wei received the B.E. degree from the School of Transportation, Shandong University of Science and Technology, China, in 2023. She is currently pursuing the master's degree with the School of Civil Aviation, Nanjing University of Aeronautics and Astronautics. Her research interests include network communications and computing.



Qingsong Liu Qingsong Liu graduated in 2024 from Nanjing University of Aeronautics and Astronautics with a bachelor's degree in Transportation from the College of Civil Aviation. He is currently pursuing a master's degree at the same university. His research interests include computer vision and video anomaly detection.



Yan Gu Yan Gu received the B.E. degree from the College of Transport and Communications, Shanghai Maritime University, China, in 2024, and is currently pursuing the master's degree with the College of Civil Aviation, Nanjing University of Aeronautics and Astronautics. Her research interests include video anomaly detection and image recognition.



Fu Xiao Fu Xiao received the Ph.D. degree in computer science and technology from the Nanjing University of Science and Technology, Nanjing, China, in 2007. He is currently a Professor and a Ph.D. supervisor with the School of Computer, Nanjing University of Posts and Telecommunications. His research interests include computer networks, data centers, and the Internet of Things. He is a member of the IEEE Computer Society and the Association for Computing Machinery.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on September 08,2026 at 12:35:58 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.